.NET Technologies

Complete Exam Preparation — Model Answers

21 Long-Answer Questions (7/8 Marks Each)
With Diagrams, Comparison Tables & Code Examples

Prepared by Arpan Adhikari

notes.arpanadhikari7.com.np

notes.arpanadhikari7.com.np

Q1. Explain the architecture of the .NET platform in detail. [8 Marks]

Answer:

The .NET platform is a software development framework developed by Microsoft that provides a managed, language-independent, and secure environment for building and executing applications (desktop, web, mobile, and cloud). Its architecture is layered: source code written in any .NET language is compiled to a common Intermediate Language (IL), which is executed by the Common Language Runtime (CLR) on top of the operating system.

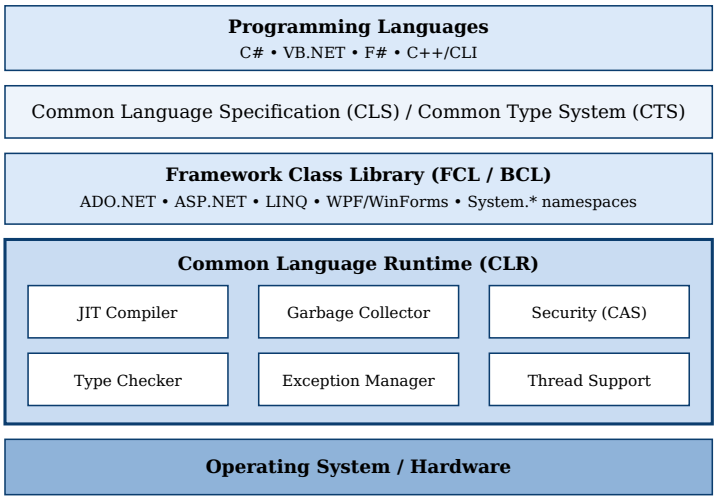


Fig 1.1: Layered architecture of the .NET platform

Main Components

1. **Common Language Runtime (CLR):** The execution engine (virtual machine) of .NET. It loads assemblies, converts IL to native machine code through the JIT compiler, and provides run-time services such as automatic memory management (Garbage Collection), type safety, exception handling, thread management, and security.
2. **Common Type System (CTS):** Defines how types (value types and reference types) are declared and used so that objects written in different .NET languages can interact. E.g., `int` in C# and `Integer` in VB.NET both map to `System.Int32`.
3. **Common Language Specification (CLS):** A subset of CTS rules that all .NET languages must follow to guarantee cross-language interoperability.
4. **Framework Class Library (FCL) / Base Class Library (BCL):** A large collection of reusable classes, interfaces, and value types (collections, I/O, networking, data access via ADO.NET, XML, LINQ) organised into namespaces such as `System`, `System.IO`, and `System.Data`.
5. **Assemblies and Metadata:** Compiled code is packaged into assemblies (`.dll`/`.exe`) containing IL code, metadata, and a manifest describing version, culture, and referenced assemblies. Metadata makes assemblies self-describing.
6. **Application Models:** On top of the FCL sit the technologies used to build applications — ASP.NET (web), ADO.NET/EF (data), WinForms/WPF (desktop), and .NET MAUI (mobile).

notes.arpanadhikari7.com.np

Execution Flow

Source code (C#/VB/F#) → language compiler → MSIL (CIL) + metadata inside an assembly → CLR loads assembly → JIT compiler converts IL to native code → native code executes under CLR supervision (managed code). This two-step compilation is what gives .NET its language independence and platform portability.

Advantages of this Architecture

- Language interoperability and a consistent programming model.
- Automatic memory management and type safety (fewer memory leaks and crashes).
- Security through code verification and controlled execution.
- Simplified deployment and versioning through self-describing assemblies.

Available on notes.arpanadhikari7.com.np

Q2. Compare .NET Framework, .NET Core and the unified .NET platforms in terms of runtime, tooling and application models. [8 Marks]

Answer:

.NET Framework (2002) is the original Windows-only implementation of .NET. **.NET Core** (2016) was a complete cross-platform, open-source redesign. From **.NET 5 (2020)** onwards, Microsoft unified both into a single platform, simply called **.NET** (currently .NET 8/9), which is the future of the ecosystem.

Aspect	.NET Framework	.NET Core	Unified .NET (5+)
Release / Status	2002; maintenance only (last version 4.8.x)	2016; superseded by .NET 5	2020–present; actively developed (LTS every 2 years)
Platform support	Windows only	Windows, Linux, macOS	Windows, Linux, macOS + mobile (iOS/Android via MAUI), WebAssembly (Blazor)
Runtime	CLR (Windows-tied); machine-wide install in GAC	CoreCLR — lightweight, modular, side-by-side versions, self-contained deployment	CoreCLR + Mono/NativeAOT; tiered JIT, ReadyToRun, AOT compilation
Library model	Monolithic FCL installed with OS	CoreFX delivered as NuGet packages	Single BCL; .NET Standard no longer needed for new work
Tooling	Visual Studio, MSBuild; no cross-platform CLI	`dotnet` CLI, VS, VS Code; SDK-style csproj	Same modern CLI/SDK; hot reload, minimal APIs, containers first-class
Application models	ASP.NET Web Forms, MVC 5, WCF, WPF, WinForms	ASP.NET Core (MVC + Web API unified), console; WPF/WinForms added in 3.0 (Windows only)	ASP.NET Core, Blazor, gRPC, Minimal APIs, MAUI, worker services, Azure Functions
Performance	Good, but heavier	Very high (Kestrel server, Span<T>)	Best-in-class; continuous improvements per release
Open source	Partially (reference source)	Fully open source (MIT)	Fully open source (MIT)
Not supported	—	Web Forms, WCF hosting, .NET Remoting	Web Forms, WCF server (use gRPC/CoreWCF)

Runtime Comparison (explanation)

The .NET Framework CLR is installed machine-wide, so all applications share one runtime version, causing upgrade conflicts. CoreCLR (used by .NET Core and unified .NET) supports **side-by-side versioning** and **self-contained deployment**, where the runtime ships with the app. The unified platform adds advanced options such as tiered compilation, ReadyToRun images, and Native AOT for fast startup.

Tooling Comparison (explanation)

.NET Framework development is tied to Visual Studio and MSBuild on Windows. .NET Core introduced the cross-platform `dotnet` CLI (`dotnet new`, `build`, `run`, `publish`, `test`) and lightweight SDK-style project files, enabling development in VS Code on any OS and easy CI/CD and Docker integration — all carried forward and enhanced in unified .NET.

Application Model Comparison (explanation)

Legacy models (Web Forms, WCF) remain only on .NET Framework. .NET Core merged MVC and Web API into ASP.NET Core. Unified .NET broadens this with Blazor (C# in the browser), gRPC for high-performance services, and .NET MAUI for cross-platform desktop/mobile from a single code base.

Conclusion: .NET Framework is legacy for existing Windows applications; .NET Core was the transitional cross-platform rewrite; unified .NET (5+) is the single, modern, high-performance platform recommended for all new development.

Available on notes.arpanadhikari7.com.np

Q3. Describe the principles of Just-In-Time (JIT) compilation. Enumerate and distinguish among its types. [8 Marks]

Answer:

Principle of JIT Compilation

In .NET, source code is not compiled directly to machine code. A language compiler first produces **Microsoft Intermediate Language (MSIL/CIL)** plus metadata inside an assembly. At run time, the CLR's **Just-In-Time (JIT) compiler** translates this IL into native machine code specific to the CPU on which the program is running, *just before* the code is executed. Compiled methods are cached in memory, so each method is JIT-compiled only once per process; subsequent calls execute the cached native code directly.

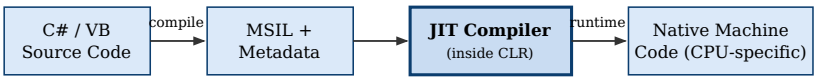


Fig 3.1: Two-stage compilation model with JIT

Key Principles / Working

- **On-demand (lazy) compilation:** Only methods that are actually invoked are compiled — unused code is never translated, saving time and memory.
- **Caching:** The native code of a compiled method is stored in memory for the lifetime of the process and reused on every subsequent call.
- **Verification and optimisation:** During JIT, the CLR verifies type safety and applies CPU-specific optimisations (register allocation, inlining), which a static compiler targeting a generic CPU cannot fully exploit.
- **Portability:** Because IL is platform-neutral, the same assembly can run on any OS/CPU that has a CLR with a suitable JIT.

Types of JIT Compilers

Type	How it works	Advantages	Disadvantages
1. Pre-JIT (AOT)	Compiles the *entire* assembly IL to native code in a single cycle at deployment/install time using `Ngen.exe` (or ReadyToRun / Native AOT in modern .NET).	Fastest startup; no run-time compilation cost.	Larger native images; loses run-time CPU-specific optimisation; must be regenerated when the assembly changes.
2. Normal JIT (Standard)	Default mode. Compiles a method the *first time it is called* and caches the native code for reuse.	Compiles only what is used; cached code makes later calls fast; full optimisation.	Slight delay ("warm-up") on the first call of each method.
3. Econo-JIT	Compiles called methods quickly with *minimal optimisation* and may *discard* the native code when memory is low, recompiling if needed again.	Very low memory footprint; fast compilation — suited to constrained devices.	No aggressive optimisation; possible recompilation cost. (Obsolete in modern .NET, replaced by tiered compilation.)

Note (modern .NET): .NET Core 3.0+ uses **Tiered Compilation**, combining these ideas — methods are first compiled quickly with low optimisation (Tier 0) and hot methods are recompiled with full optimisation (Tier 1), giving both fast startup and high steady-state performance.

Distinction summary: Pre-JIT compiles *everything before execution*; Normal JIT compiles *per method at first call with optimisation and permanent caching*; Econo-JIT compiles *per method with minimal optimisation and temporary caching*.

Available on notes.arpanadhikari7.com.np

Q4. What is a Web API? What are the advantages of using Web API? [7 Marks]

Answer:

Definition

A **Web API** (Application Programming Interface) is an HTTP-based service framework that exposes application data and functionality over the web so that a wide range of clients — browsers, mobile apps, desktop apps, IoT devices, and other servers — can consume it. In .NET, **ASP.NET Web API / ASP.NET Core Web API** is used to build **RESTful services** that use standard HTTP verbs (GET, POST, PUT, DELETE) on resources identified by URLs and typically exchange data in **JSON** or XML format.

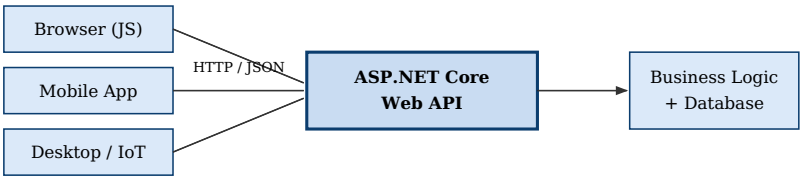


Fig 4.1: Multiple heterogeneous clients consuming one Web API over HTTP

Key Characteristics

- Built on HTTP — uses verbs, status codes (200, 404, 500), headers, and content negotiation.
- Stateless request/response model following REST principles.
- Returns data (JSON/XML), not HTML views — presentation is left to the client.
- In ASP.NET Core, controllers derive from `ControllerBase` and use attribute routing, e.g. `[HttpGet("api/students/{id}")]`.

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpGet("{id}")]
    public ActionResult<Student> Get(int id) => _repo.Find(id) is Student s ? s : NotFound();
}
```

Advantages of Web API

1. **Platform and client independence:** Any client capable of HTTP (web, Android, iOS, IoT) can consume it — one back end serves all front ends.
2. **Lightweight and fast:** Uses plain HTTP + JSON instead of heavy SOAP/XML envelopes, reducing bandwidth — ideal for mobile devices with limited resources.
3. **Full HTTP feature support:** Caching, content negotiation (JSON/XML), status codes, versioning, and request/response headers.
4. **Statelessness and scalability:** Stateless REST calls can be load-balanced across servers easily, enabling horizontal scaling and microservices architectures.
5. **Easy integration:** Enables third-party integration and public APIs (e.g., payment gateways, maps) and supports AJAX/SPA frameworks (Angular, React) naturally.
6. **Built-in framework features:** Model binding, validation, routing, filters, dependency injection, and security (JWT/OAuth) come out of the box in ASP.NET Core.
7. **Testability and tooling:** Endpoints can be tested with tools like Postman/Swagger (OpenAPI) and unit-tested independently of any UI.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Q5. Distinguish between abstract classes and interfaces in .NET. Provide use cases where one is preferred over the other. [8 Marks]

Answer:

Both abstract classes and interfaces are used to achieve **abstraction** and **polymorphism** in .NET, but they differ fundamentally in purpose: an abstract class represents an **"is-a"** relationship with shared implementation, whereas an interface represents a **"can-do"** contract of capabilities.

Basis	Abstract Class	Interface
Keyword	Declared with `abstract class`	Declared with `interface`
Implementation	Can contain both abstract members and fully implemented (concrete) methods, fields, and constructors	Traditionally only declarations; C# 8+ allows default implementations, but no instance fields or constructors
Inheritance	A class can inherit only **one** abstract class (single inheritance)	A class can implement **multiple** interfaces
Members allowed	Fields, properties, methods, events, constructors, destructors; any access modifier	Methods, properties, events, indexers; members are public by default; no instance fields
State	Can hold state (instance variables)	Cannot hold instance state
Instantiation	Cannot be instantiated directly	Cannot be instantiated directly
Versioning	Adding a new concrete method does not break derived classes	Adding a member breaks all implementers (unless a default implementation is given)
Speed	Slightly faster member access	Slight overhead due to interface dispatch
Relationship	"is-a" (Dog *is an* Animal)	"can-do" (Dog *can* IComparable)

Code Illustration

```
public abstract class Shape                // abstract class: shared code + contract
{
    public string Name { get; set; }       // state allowed
    public abstract double Area();         // must be overridden
    public void Describe() =>              // concrete shared behaviour
        Console.WriteLine($"{Name} area = {Area()}");
}

public interface IDrawable { void Draw(); } // interface: pure capability
public interface IResizable { void Resize(double f); }

public class Circle : Shape, IDrawable, IResizable   // one class, many interfaces
{
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
    public void Draw() => Console.WriteLine("Drawing circle");
    public void Resize(double f) => Radius *= f;
}
```

notes.arpanadhikari7.com.np

When to Prefer an Abstract Class

- When several closely related classes must **share common code and state** (e.g., `Shape` base with a `Name` field and a common `Describe()` method).
- When you need **constructors, fields, or non-public members** in the base type.
- When you expect the base to **evolve** — new concrete methods can be added without breaking subclasses (e.g., ASP.NET Core's `ControllerBase`, ADO.NET's `DbConnection`).

When to Prefer an Interface

- When **unrelated classes** need a common capability (e.g., `IComparable`, `IDisposable`, `IEnumerable` are implemented by hundreds of unrelated types).
- When a class needs **multiple inheritance of behaviour contracts** (C# forbids multiple base classes but allows many interfaces).
- For **dependency injection and unit testing** — services are registered and mocked via interfaces (e.g., `IRepository`, `ILogger`) to achieve loose coupling.
- When defining a **plug-in or API contract** where implementers should not inherit any implementation.

Conclusion: Use an abstract class to share implementation among a family of related types; use an interface to define a contract that many unrelated types can fulfil and to keep components loosely coupled.

Available on notes.arpanadhikari7.com.np

Q6. Explain the middleware pipeline architecture in ASP.NET Core. How does it enhance modular development? [8 Marks]

Answer:

Concept

Middleware in ASP.NET Core is software assembled into an application **pipeline** to handle HTTP requests and responses. Each middleware component: (i) receives the incoming `HttpContext`, (ii) can perform work *before* passing control to the next component via the `next()` delegate, (iii) can **short-circuit** the pipeline (return a response immediately), and (iv) can perform work *after* the next component returns — so the response flows back through the same components in reverse order.

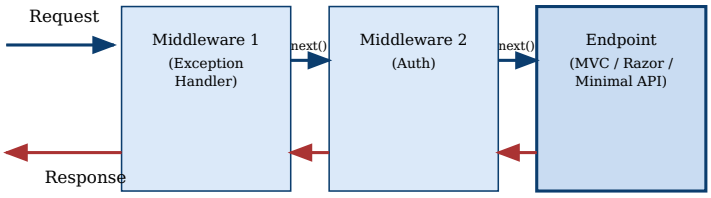


Fig 6.1: Request flows forward through middleware; response flows back in reverse

Typical order: ExceptionHandler → HTTPS → StaticFiles → Routing → CORS → AuthN → AuthZ → Endpoints

Configuring the Pipeline (Program.cs)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();          // register services (DI)
var app = builder.Build();

app.UseExceptionHandler("/error");          // 1. catches errors from later middleware
app.UseHttpsRedirection();
app.UseStaticFiles();                       // may short-circuit for static content
app.UseRouting();
app.UseAuthentication();                    // who are you?
app.UseAuthorization();                     // what may you do?

app.Use(async (context, next) =>            // custom inline middleware
{
    Console.WriteLine($"IN : {context.Request.Path}");
    await next();                           // call next middleware
    Console.WriteLine($"OUT: {context.Response.StatusCode}");
});

app.MapControllers();                       // terminal endpoint middleware
app.Run();
```

Key Methods

- `Use()` — adds middleware that can call the next component (chainable).
- `Run()` — adds *terminal* middleware that ends the pipeline.
- `Map()` / `MapWhen()` — branches the pipeline based on request path/condition.

notes.arpanadhikari7.com.np

How It Enhances Modular Development

1. **Single responsibility:** Each cross-cutting concern (logging, authentication, CORS, compression, error handling) lives in its own small, independent component instead of one monolithic handler (unlike the fixed HttpModules/Handlers of classic ASP.NET).
2. **Composability & explicit ordering:** The developer chooses exactly which components run and in what order — the app carries no unused features, improving performance.
3. **Reusability:** Middleware is packaged as classes/NuGet packages and reused across projects (`UseSwagger()`, `UseCors()`).
4. **Extensibility:** Custom middleware is trivial to write (a class with an `InvokeAsync(HttpContext, RequestDelegate)` method) and plug in anywhere.
5. **Short-circuiting:** Requests can be answered early (static files, cached responses, failed auth), saving downstream processing.
6. **Testability:** Each middleware can be unit-tested in isolation, and DI supplies its dependencies.

Available on notes.arpanadhikari7.com.np

Q7. Explain the ASP.NET page life cycle. Detail the key events during its execution.

[8 Marks]

Answer:

In ASP.NET Web Forms, every request for an `.aspx` page causes the runtime to build the page object, run it through a well-defined **life cycle** of stages and events, render HTML, and dispose of the page. Understanding the cycle is essential for placing code correctly (e.g., initialising controls vs. reading posted data).

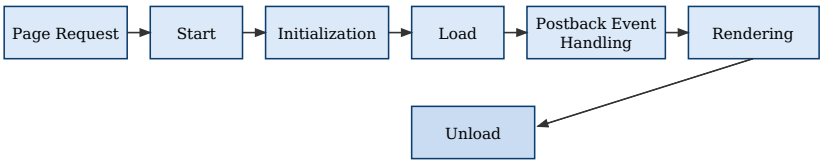


Fig 7.1: Stages of the ASP.NET Web Forms page life cycle

Key Events in Order of Execution

#	Event	Purpose / Typical Use
1	**PreInit**	First event. Check `IsPostBack`; set master page, theme, or create dynamic controls; ViewState is *not* yet loaded.
2	**Init**	Each control is initialised (fires child-first, then page). Control properties can be set; ViewState still not available.
3	**InitComplete**	All initialisation done; ViewState tracking is turned on.
4	**PreLoad**	Fires after ViewState and postback data have been loaded into controls, just before Load.
5	**Load** (`Page_Load`)	Most-used event. Page and all controls are loaded; connect to databases, bind data, read control values. Guard one-time logic with `if (!IsPostBack)`.
6	**Control (Postback) events**	Server-side events raised by user actions — e.g., `Button_Click`, `SelectedIndexChanged`, and Validation occurs here.
7	**LoadComplete**	All controls loaded and postback events processed; suitable for tasks needing every control ready.
8	**PreRender**	Last chance to change page/control content before HTML is generated (final data binding, last-minute changes).
9	**SaveStateComplete**	ViewState/control state has been serialised into the hidden `__VIEWSTATE` field; changes to controls no longer persist.
10	**Render**	Not an event but a method — each control writes its HTML markup to the response stream.
11	**Unload**	Page sent to client; perform cleanup — close files/DB connections, release resources. Response stream can no longer be modified.

```
protected void Page_Load(object sender, EventArgs e)
{
    if (!IsPostBack)          // run only on first GET, not on postbacks
    {
        GridView1.DataSource = GetStudents();
        GridView1.DataBind();
    }
}
```

notes.arpanadhikari7.com.np

Postback note: On a postback, the same cycle runs again, but ViewState and posted form data restore control values between Init and Load, and the triggering control's event (e.g., Click) fires after Load.

Available on notes.arpanadhikari7.com.np

Q8. Describe the authentication and authorization mechanisms in ASP.NET. [8 Marks]

Answer:

Authentication answers "*Who are you?*" — verifying a user's identity (credentials, token, certificate). **Authorization** answers "*What are you allowed to do?*" — deciding whether the authenticated identity may access a resource. Authentication always occurs first; in the ASP.NET Core pipeline this is enforced by placing `UseAuthentication()` before `UseAuthorization()`.

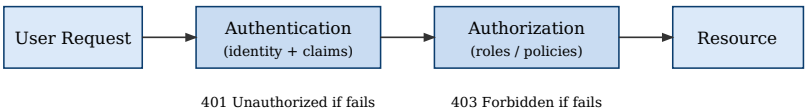


Fig 8.1: Security pipeline — authenticate, then authorize

Authentication Mechanisms

1. **Forms Authentication (classic ASP.NET):** Unauthenticated users are redirected to a login page; on success an encrypted authentication ticket is stored in a cookie and validated on each request. Configured in `web.config` (`<authentication mode="Forms">`).
2. **Windows Authentication:** Uses the user's Windows/Active Directory account via NTLM or Kerberos — ideal for intranet applications; no separate login page needed.
3. **Passport / external authentication:** Centralised Microsoft sign-in (legacy); replaced today by **OAuth 2.0 / OpenID Connect** external providers (Google, Facebook, Microsoft, Azure AD).
4. **ASP.NET Core Identity:** A complete membership system — user/role stores in a database via EF Core, password hashing, lockout, two-factor authentication, and external logins.
5. **Cookie Authentication (ASP.NET Core):** After login a signed cookie carries the user's `ClaimsPrincipal` on each request — the default for browser apps.
6. **JWT Bearer Token Authentication:** For Web APIs/SPAs/mobile. The server issues a signed JSON Web Token; the client sends it in the `Authorization: Bearer <token>` header; being stateless, it scales well and suits microservices.

Authorization Mechanisms

1. **Simple authorization:** `[Authorize]` on a controller/action requires any authenticated user; `[AllowAnonymous]` opts out.
2. **Role-based authorization:** Access based on membership in roles — `[Authorize(Roles = "Admin,Manager")]`. In classic ASP.NET this maps to URL authorization: `<allow roles="Admin"/> <deny users="*"/>`.
3. **Claims-based authorization:** Decisions based on claims (key–value facts about the user such as Department = "IT") contained in the identity.
4. **Policy-based authorization (ASP.NET Core):** Named policies combine requirements and custom handlers:

```
builder.Services.AddAuthorization(o =>
    o.AddPolicy("Adult", p => p.RequireClaim("Age").RequireAssertion(
        ctx => int.Parse(ctx.User.FindFirst("Age")!.Value) >= 18)));
// usage:  [Authorize(Policy = "Adult")]
```

notes.arpanadhikari7.com.np

5. **Resource-based authorization:** Imperative checks against a specific resource (e.g., only the document's owner may edit it) using `IAuthorizationService`.

Summary: Authentication establishes identity (Forms, Windows, Identity, cookies, JWT/OAuth); authorization then grants or denies access using roles, claims, or policies — together forming the layered security model of ASP.NET.

Available on notes.arpanadhikari7.com.np

Q9. What is a connection string in .NET? List and describe the key components of a typical SQL Server connection string. [7 Marks]

Answer:

Definition

A **connection string** is a string of semicolon-separated `key=value` pairs that tells a data provider (e.g., ADO.NET's `SqlConnection` or EF Core) everything it needs to locate and open a connection to a database: the server, the database name, authentication credentials, and optional behaviour settings (timeout, encryption, pooling). It is normally stored outside the code — in `web.config` / `app.config` (`<connectionStrings>`) or in `appsettings.json` ("ConnectionStrings" section) in ASP.NET Core — so it can be changed per environment without recompiling.

```
// appsettings.json
"ConnectionStrings": {
  "Default": "Server=DBSRV01;Database=StudentDB;User ID=appuser;Password=P@ss1;
              Encrypt=True;TrustServerCertificate=False;Connect Timeout=30;"
}

// usage with ADO.NET
using var con = new SqlConnection(builder.Configuration.GetConnectionString("Default"));
con.Open();
```

Key Components of a SQL Server Connection String

Component (aliases)	Description	Example
Server (Data Source, Addr)	Name/IP of the SQL Server instance, optionally with instance name or port.	`Server=DBSRV01\SQLEXPRESS`, `Server=10.0.0.5,1433`, `.` or `(localdb)\MSSQLLocalDB`
Database (Initial Catalog)	The default database to open on the server.	`Database=StudentDB`
Integrated Security (Trusted_Connection)	`True/SSPI` = use the current Windows account (no username/password needed); `False` = SQL authentication.	`Integrated Security=SSPI`
User ID (uid)	SQL Server login name when SQL authentication is used.	`User ID=appuser`
Password (pwd)	Password for the SQL login. Should be protected (secrets store / environment variables), never hard-coded.	`Password=P@ss1`
Connect Timeout (Connection Timeout)	Seconds to wait while trying to establish the connection before throwing an error (default 15).	`Connect Timeout=30`
Encrypt	Whether TLS/SSL encryption is used for data in transit.	`Encrypt=True`
TrustServerCertificate	If `True`, skips validation of the server's TLS certificate (use only in development).	`TrustServerCertificate=False`
Pooling / Min–Max Pool Size	Enables connection pooling (default on) and sets pool limits, greatly improving performance by reusing open connections.	`Pooling=True;Max Pool Size=100`
MultipleActiveResultSets (MARS)	Allows multiple simultaneous open readers/commands on one connection.	`MultipleActiveResultSets=True`
Application Name	Identifies the application in SQL Server monitoring/profiling tools.	`Application Name=StudentApp`
AttachDbFilename	Attaches an `.mdf` database file directly (LocalDB scenarios).	`AttachDbFilename=|DataDirectory|\db.mdf`

notes.arpanadhikari7.com.np

Best Practices

- Store connection strings in configuration (appsettings.json / user secrets / environment variables / Azure Key Vault), never in source code.
- Prefer Integrated Security or managed identities over embedded passwords where possible.
- Always dispose connections (`using` block) so they return to the pool promptly.

Q10. Discuss the concept and benefits of dependency injection in .NET Core with an implementation example. [8 Marks]

Answer:

Concept

Dependency Injection (DI) is a design pattern implementing the **Inversion of Control (IoC)** principle: instead of a class creating its own dependencies with `new`, the dependencies are declared (usually as interfaces) and are **supplied ("injected") from outside** by a container. ASP.NET Core has a built-in IoC container: services are *registered* in `Program.cs`, and the container *resolves and injects* them automatically — most commonly through **constructor injection**.

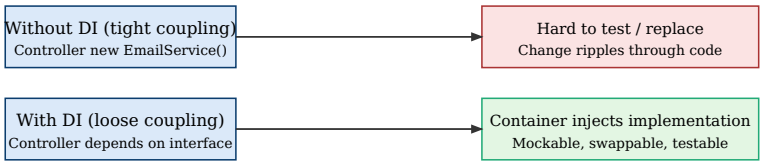


Fig 10.1: DI decouples consumers from concrete implementations

Service Lifetimes

Lifetime	Registration	Instance created	Typical use
Transient	`AddTransient<I,T>()`	Every time it is requested	Lightweight, stateless services
Scoped	`AddScoped<I,T>()`	Once per HTTP request	DbContext, unit-of-work, repositories
Singleton	`AddSingleton<I,T>()`	Once for the whole application	Caches, configuration, loggers

Implementation Example

```
// 1. Abstraction
public interface IMessageService { void Send(string to, string msg); }

// 2. Implementation
public class EmailService : IMessageService
{
    public void Send(string to, string msg) =>
        Console.WriteLine($"Email to {to}: {msg}");
}

// 3. Registration (Program.cs) — the composition root
builder.Services.AddScoped<IMessageService, EmailService>();

// 4. Constructor injection into a controller
public class NotifyController : Controller
{
    private readonly IMessageService _svc;
    public NotifyController(IMessageService svc)  // container supplies EmailService
    {
        _svc = svc;
    }

    public IActionResult Index()
    {
        _svc.Send("student@tsu.edu", "Result published");
        return View();
    }
}
```

notes.arpanadhikari7.com.np

To switch to SMS delivery later, only the registration changes (`AddScoped<IMessageService, SmsService>()`) — the controller code is untouched.

Benefits of Dependency Injection

1. **Loose coupling:** Classes depend on abstractions, not concrete types (Dependency Inversion Principle of SOLID).
2. **Testability:** Real services can be replaced by mocks/stubs in unit tests (e.g., a fake `IMessageService`).
3. **Maintainability & flexibility:** Implementations can be swapped or upgraded in one place without modifying consumers.
4. **Centralised lifetime management:** The container creates and disposes objects correctly (e.g., one DbContext per request).
5. **Cleaner code & reuse:** No scattered `new` statements; cross-cutting services (logging, config, caching) are

Available on notes.arpanadhikari7.com.np

shared consistently.

notes.arpanadhikari7.com.np

Q11. Explain the role and structure of appsettings.json in ASP.NET Core projects. How does it support configuration management? [7 Marks]

Answer:

Role

`appsettings.json` is the primary, human-readable **configuration file** of an ASP.NET Core application. It replaces the XML-based `web.config` of classic ASP.NET for application settings, storing values such as connection strings, logging levels, feature flags, API keys, and custom application options as structured **JSON**. It is loaded automatically at startup by the host into the unified `IConfiguration` system.

Typical Structure

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "Default": "Server=.;Database=StudentDB;Trusted_Connection=True;"
  },
  "AllowedHosts": "*",
  "AppSettings": {                      // custom section
    "AppName": "Student Portal",
    "MaxPageSize": 50,
    "Smtp": { "Host": "smtp.tsu.edu", "Port": 587 }
  }
}
```

The file is **hierarchical**: nested objects form sections, and any value is addressed by a colon-separated path, e.g. `"AppSettings:Smtp:Host"`.

Reading Configuration Values

```
// (a) Direct access via IConfiguration (injected)
public HomeController(IConfiguration config)
{
    string name = config["AppSettings:AppName"];
    string cs   = config.GetConnectionString("Default");
}

// (b) Strongly typed Options pattern (recommended)
public class AppSettings { public string AppName { get; set; } public int MaxPageSize { get; set; } }

builder.Services.Configure<AppSettings>(
    builder.Configuration.GetSection("AppSettings"));

public class ReportService
{
    public ReportService(IOptions<AppSettings> opt)
        => Console.WriteLine(opt.Value.AppName);   // "Student Portal"
}
```

notes.arpanadhikari7.com.np

How It Supports Configuration Management

1. **Environment-specific overrides:** `appsettings.{Environment}.json` files (e.g., `appsettings.Development.json`, `appsettings.Production.json`) are layered on top of the base file according to `ASPNETCORE_ENVIRONMENT`, so each environment can override only the values that differ — no code changes or recompilation needed.
2. **Layered provider model:** It is one of several configuration providers loaded in order — appsettings.json → environment JSON → user secrets (dev) → environment variables → command-line arguments — where **later sources override earlier ones**, enabling secure production overrides (secrets stay out of source control).
3. **Strong typing and validation:** The Options pattern binds sections to POCO classes, giving IntelliSense, compile-time safety, and validation.
4. **Change reload:** With `reloadOnChange: true` (default) and `IOptionsSnapshot` / `IOptionsMonitor`, edited settings can be picked up without restarting the app.
5. **Centralisation and DI integration:** All settings live in one predictable place and flow to any class through dependency injection, keeping configuration concerns separate from business logic.

Available on notes.arpanadhikari7.com.np

Q12. How can you perform CRUD operations using Entity Framework? Provide code examples for each operation. [8 Marks]

Answer:

Entity Framework (EF/EF Core) is Microsoft's Object-Relational Mapper (ORM). CRUD — **C**reate, **R**ead, **U**pdate, **D**elete — operations are performed on C# entity objects through a `DbContext`; EF translates them into SQL (`INSERT`, `SELECT`, `UPDATE`, `DELETE`) and executes them on `SaveChanges()`. The context tracks entity states (*Added, Unchanged, Modified, Deleted*) to know which SQL to generate.

Setup: Entity and DbContext

```
public class Student
{
    public int Id { get; set; }              // primary key by convention
    public string Name { get; set; }
    public string Course { get; set; }
    public double GPA { get; set; }
}

public class SchoolContext : DbContext
{
    public SchoolContext(DbContextOptions<SchoolContext> o) : base(o) { }
    public DbSet<Student> Students { get; set; }   // maps to Students table
}

// Program.cs registration
builder.Services.AddDbContext<SchoolContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
```

1. CREATE — insert a new record

```
using (var db = new SchoolContext(options))
{
    var s = new Student { Name = "Akriti", Course = "MSc CS", GPA = 3.9 };
    db.Students.Add(s);        // state = Added
    db.SaveChanges();          // SQL: INSERT INTO Students (...) VALUES (...)
    Console.WriteLine($"New Id = {s.Id}");   // identity value populated back
}
```

2. READ — retrieve records (LINQ queries)

```
using (var db = new SchoolContext(options))
{
    Student one   = db.Students.Find(1);                    // by primary key
    Student first = db.Students.FirstOrDefault(s => s.Name == "Akriti");
    List<Student> toppers = db.Students
                              .Where(s => s.GPA >= 3.5)     // filtering
                              .OrderByDescending(s => s.GPA)
                              .ToList();                    // SQL: SELECT ... WHERE ...
}
```

3. UPDATE — modify an existing record

notes.arpanadhikari7.com.np

```
using (var db = new SchoolContext(options))
{
    var s = db.Students.Find(1);   // fetch (tracked entity)
    if (s != null)
    {
        s.GPA = 4.0;               // state becomes Modified
        s.Course = "PhD Entomology";
        db.SaveChanges();          // SQL: UPDATE Students SET ... WHERE Id=1
    }
}
```

4. DELETE — remove a record

```
using (var db = new SchoolContext(options))
{
    var s = db.Students.Find(2);
    if (s != null)
    {
        db.Students.Remove(s);     // state = Deleted
        db.SaveChanges();          // SQL: DELETE FROM Students WHERE Id=2
    }
}
```

CRUD in an ASP.NET Core Web API Controller (async)

```
[ApiController]
[Route("api/[controller]")]
```

Available on notes.arpanadhikari7.com.np

```
public class StudentsController : ControllerBase
{
    private readonly SchoolContext _db;
    public StudentsController(SchoolContext db) => _db = db;

    [HttpGet]                                   // READ all
    public async Task<IEnumerable<Student>> GetAll() => await _db.Students.ToListAsync();

    [HttpPost]                                  // CREATE
    public async Task<IActionResult> Create(Student s)
    { _db.Students.Add(s); await _db.SaveChangesAsync();
      return CreatedAtAction(nameof(GetAll), new { id = s.Id }, s); }

    [HttpPut("{id}")]                           // UPDATE
    public async Task<IActionResult> Update(int id, Student s)
    { if (id != s.Id) return BadRequest();
      _db.Entry(s).State = EntityState.Modified;
      await _db.SaveChangesAsync(); return NoContent(); }

    [HttpDelete("{id}")]                        // DELETE
    public async Task<IActionResult> Delete(int id)
    { var s = await _db.Students.FindAsync(id);
      if (s is null) return NotFound();
      _db.Students.Remove(s); await _db.SaveChangesAsync(); return NoContent(); }
}
```

Key point: All changes are batched in the change tracker and committed atomically by `SaveChanges()` / `SaveChangesAsync()`, which wraps them in a database transaction.

notes.arpanadhikari7.com.np

Q13. Discuss the role and advantages of Entity Framework Core in data access operations.

[7 Marks]

Answer:

Role of EF Core

Entity Framework Core is the modern, lightweight, cross-platform **Object-Relational Mapper (ORM)** for .NET. Its role is to bridge the "impedance mismatch" between object-oriented C# code and relational databases: developers work with **entity classes** and **LINQ queries**, while EF Core generates and executes the SQL, materialises result rows back into objects, tracks changes, and manages transactions. It thus serves as the **data access layer** that removes most hand-written ADO.NET/SQL plumbing.



Fig 13.1: EF Core sits between application objects and the relational database

Core Responsibilities

- **Object–relational mapping:** Maps classes↔tables, properties↔columns, and relationships (1:1, 1:N, N:N) via conventions, data annotations, or Fluent API.
- **Query translation:** Converts LINQ expressions into provider-specific, parameterised SQL and materialises results into objects.
- **Change tracking & unit of work:** `DbContext` tracks entity states and persists all modifications atomically on `SaveChanges()`.
- **Schema management (Migrations):** `Add-Migration` / `Update-Database` evolve the database schema from the model, version-controlled with the code (Code-First); Database-First scaffolding is also supported.

Advantages of EF Core

1. **Higher productivity, less boilerplate:** No manual connection/command/reader code or hand-written SQL for routine operations — CRUD becomes a few lines of C#.
2. **Strong typing & compile-time checking:** LINQ queries are checked by the compiler and supported by IntelliSense, catching errors that raw SQL strings would hide.
3. **Database independence:** Providers exist for SQL Server, SQLite, PostgreSQL, MySQL, Oracle, Cosmos DB, and the InMemory provider — switching databases mainly means changing the provider and connection string.
4. **Security:** All generated queries are parameterised, protecting against SQL injection by default.
5. **Migrations:** Safe, repeatable, versioned schema evolution synchronized with the entity model across dev/test/production.
6. **Performance features:** Compiled queries, no-tracking queries (`AsNoTracking()`), eager/lazy/explicit loading control, batching of statements, and full async support (`ToListAsync`, `SaveChangesAsync`) for scalable web apps.
7. **Testability & maintainability:** The InMemory/SQLite providers enable fast unit tests; centralised mapping keeps data access consistent and easy to refactor; integrates naturally with DI and the Repository pattern.
8. **Cross-platform & open source:** Runs on Windows/Linux/macOS under .NET, actively developed on GitHub.

Limitation note (for balance): For extremely performance-critical or very complex queries, developers can still drop to raw SQL (`FromSqlInterpolated`) or stored procedures — EF Core allows mixing both approaches.

Available on notes.arpanadhikari7.com.np

Q14. Describe structured exception handling in C# using try, catch, and finally constructs. [7 Marks]

Answer:

Structured Exception Handling (SEH) is C#'s mechanism for detecting and responding to run-time errors (exceptions) in a controlled way, instead of letting the program crash. An **exception** is an object derived from `System.Exception` that is *thrown* when an abnormal condition occurs (divide by zero, null reference, file missing) and *caught* by a matching handler.

The Constructs

- **`try`** — encloses the "guarded" code that may throw an exception. Must be followed by at least one `catch` or a `finally`.
- **`catch`** — handles a specific exception type. Multiple catch blocks are checked top-down; the **first matching (most derived first)** block runs. Exception filters (`when`) allow conditional catching.
- **`finally`** — always executes, whether an exception occurred or not — even after a `return`. Used for cleanup: closing files, database connections, releasing locks.
- **`throw`** — raises an exception; bare `throw;` inside a catch re-throws preserving the original stack trace.

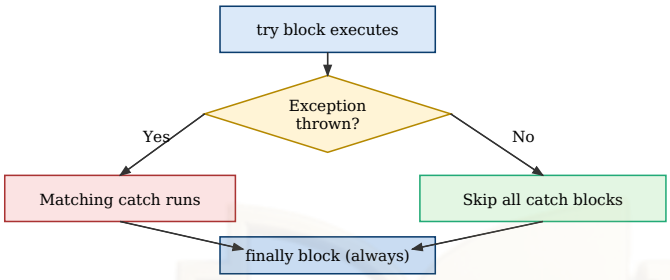


Fig 14.1: Control flow of try–catch–finally

Syntax and Example

```
try
{
    int[] marks = { 90, 85, 78 };
    Console.Write("Enter index and divisor: ");
    int idx = int.Parse(Console.ReadLine());
    int div = int.Parse(Console.ReadLine());
    Console.WriteLine(marks[idx] / div);          // may throw several exceptions
}
catch (DivideByZeroException ex)                  // most specific first
{
    Console.WriteLine("Error: cannot divide by zero. " + ex.Message);
}
catch (IndexOutOfRangeException ex)
{
    Console.WriteLine("Error: invalid index. " + ex.Message);
}
catch (FormatException ex) when (ex.Message.Contains("input"))  // exception filter
{
    Console.WriteLine("Error: enter numbers only.");
}
catch (Exception ex)                              // general handler last
{
    Console.WriteLine("Unexpected error: " + ex.Message);
    throw;                                        // re-throw, keep stack trace
}
finally
{
    Console.WriteLine("Cleanup: closing resources...");  // ALWAYS runs
}
```

Important Rules and Points

1. Catch blocks must be ordered from **most specific to most general**; a base-class catch before a derived one is a compile-time error.
2. `finally` executes even if a `return`/`break` occurs in try or catch — guaranteeing resource cleanup (the `using` statement is syntactic sugar over try/finally calling `Dispose()`).
3. If no catch matches, the exception propagates **up the call stack**; if never handled, the CLR terminates the program after running finally blocks.
4. Useful `Exception` members: `Message`, `StackTrace`, `InnerException`, `Source`.
5. Custom exceptions are created by deriving from `Exception` (e.g., `class InvalidGpaException : Exception`) for domain-specific errors.

Available on notes.arpanadhikari7.com.np

Benefits: Separates error-handling code from normal logic, prevents crashes, guarantees cleanup, preserves diagnostic information, and allows graceful recovery — making applications robust and maintainable.

notes.arpanadhikari7.com.np

Q15. What is Language Integrated Query (LINQ)? Explain how LINQ enhances data querying within .NET applications. [7 Marks]

Answer:

Definition

LINQ (Language Integrated Query), introduced in .NET 3.5 / C# 3.0, integrates query capabilities **directly into the C#/VB language**. With one consistent, SQL-like syntax, the same querying model works over many data sources through different providers: in-memory collections (**LINQ to Objects**), databases (**LINQ to SQL / LINQ to Entities** via EF), XML (**LINQ to XML**), and DataSets. Queries are built from **standard query operators** (`Where, Select, OrderBy, GroupBy, Join, Sum, Count…`) implemented as extension methods on `IEnumerable<T>` / `IQueryable<T>`, using lambda expressions.

Two Syntaxes

```
var students = new List<Student> {
    new("Akriti", "CS", 3.9), new("Bina", "Math", 3.2), new("Chandan", "CS", 3.6) };

// (a) Query syntax (SQL-like)
var q1 = from s in students
         where s.Course == "CS" && s.GPA >= 3.5
         orderby s.GPA descending
         select new { s.Name, s.GPA };

// (b) Method syntax (lambda-based) — equivalent
var q2 = students.Where(s => s.Course == "CS" && s.GPA >= 3.5)
                 .OrderByDescending(s => s.GPA)
                 .Select(s => new { s.Name, s.GPA });

// Grouping and aggregation
var byCourse = students.GroupBy(s => s.Course)
                       .Select(g => new { Course = g.Key,
                                          Count = g.Count(),
                                          AvgGpa = g.Average(s => s.GPA) });
```

How LINQ Enhances Data Querying in .NET

1. **Unified query model:** One syntax queries objects, SQL databases, XML, and more — developers no longer learn a separate query language per data source.
2. **Compile-time checking & IntelliSense:** Queries are C# code, so the compiler catches wrong column names or type mismatches that would only fail at run time in SQL strings; full IDE support (auto-completion, refactoring, debugging).
3. **Strong typing:** Results are typed objects (including anonymous types), eliminating casting from `DataReader` columns and reducing run-time errors.
4. **Declarative and readable:** Developers state *what* data they want, not *how* to loop and filter — replacing verbose foreach/if code with concise, self-documenting queries.
5. **Deferred execution:** A query is only an expression until it is enumerated (`foreach`, `ToList()`, `Count()`), allowing queries to be composed dynamically and executed with the latest data; immediate execution is available when needed (`ToList`, `ToArray`).
6. **Provider translation (IQueryable):** Against EF, the LINQ expression tree is translated into efficient, parameterised SQL executed on the server — combining C# convenience with database performance and SQL-injection safety.
7. **Powerful operators & composability:** Filtering, projection, ordering, grouping, joining, set operations, and aggregation can be chained fluently; **PLINQ** (`AsParallel()`) parallelises in-memory queries for large data.
8. **Less boilerplate, fewer bugs:** Common data-manipulation patterns shrink to one-liners, improving productivity and maintainability.

Conclusion: LINQ turns data querying from error-prone string-based or loop-based code into first-class, type-safe language constructs, dramatically improving correctness, readability, and developer productivity across all .NET data sources.

notes.arpanadhikari7.com.np

Q16. Discuss asynchronous programming in .NET with reference to async and await keywords. How do they improve application responsiveness? [8 Marks]

Answer:

Concept

Asynchronous programming allows a program to start a long-running operation (file/network I/O, database query, web request) and **continue doing other work instead of blocking** the current thread while waiting. .NET implements this through the **Task-based Asynchronous Pattern (TAP)**: asynchronous methods return `Task` / `Task<T>` (a promise of a future result), and the `async` and `await` keywords (C# 5.0) let developers write asynchronous code that reads like normal sequential code.

The async and await Keywords

- `async` — a method modifier that (i) enables the use of `await` inside the method and (ii) makes the compiler rewrite the method into a **state machine**. Signature: returns `Task`, `Task<T>`, or `void` (void only for event handlers).
- `await` — applied to a Task: if the task is already complete, execution continues immediately; otherwise the method **suspends**, **returns control to its caller** (freeing the thread), and registers a **continuation** that resumes the method when the task finishes. The awaited result is unwrapped (`T` from `Task<T>`), and exceptions propagate naturally into try/catch.

Example

```
// Synchronous (blocking) version — UI freezes / server thread wasted
public string Download() {
    var wc = new HttpClient();
    return wc.GetStringAsync(url).Result;   // blocks the thread!
}

// Asynchronous version
public async Task<int> GetPageLengthAsync(string url)
{
    using var http = new HttpClient();
    Console.WriteLine("Requesting...");
    string html = await http.GetStringAsync(url);  // thread released here
    Console.WriteLine("Received.");                // continuation resumes here
    return html.Length;
}

// Calling code
public async Task RunAsync()
{
    Task<int> t = GetPageLengthAsync("https://example.com"); // starts work
    DoOtherWork();                          // runs concurrently with download
    int length = await t;                   // wait (non-blocking) for result
    Console.WriteLine($"Length = {length}");
}

// ASP.NET Core action — scalable server code
[HttpGet]
public async Task<IEnumerable<Student>> Get() =>
    await _db.Students.ToListAsync();
```

notes.arpanadhikari7.com.np

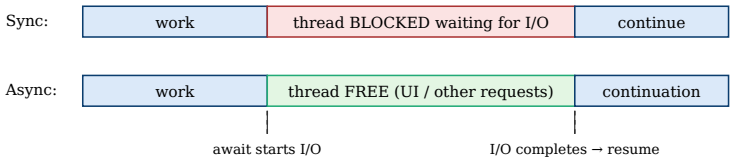


Fig 16.1: await frees the thread during I/O instead of blocking it

How async/await Improve Responsiveness

1. **Non-blocking UI (desktop/mobile):** The UI thread is released at each `await`, so windows keep repainting and responding to clicks during long operations — no "Not Responding" freezes; the continuation resumes on the UI thread automatically (SynchronizationContext).
2. **Server scalability (ASP.NET):** While a request awaits a database/HTTP call, its thread returns to the thread pool to serve **other requests**. The same server handles far more concurrent users with fewer threads — higher throughput without extra hardware.
3. **Efficient resource usage:** Threads are expensive (~1 MB stack each); async I/O uses OS completion callbacks instead of parked threads, reducing memory and context-switch overhead.

Available on notes.arpanadhikari7.com.np

4. **Natural concurrency:** Multiple tasks can be started and awaited together (`await Task.WhenAll(t1, t2)`), overlapping independent operations to cut total latency.
5. **Maintainable code:** Compared to old callbacks/APM, async/await keeps sequential structure, works with try/catch/finally, loops, and using — asynchronous code that is as readable as synchronous code.

Best practices: Use "async all the way" (avoid `.Result`/`.Wait()`, which block and can deadlock); suffix method names with `Async`; use `Task.Run` only for CPU-bound work; support `CancellationToken` for cancellable operations.

notes.arpanadhikari7.com.np

Q17. Discuss the benefits and challenges of adopting a microservices approach in enterprise .NET applications. [8 Marks]

Answer:

Concept

A **microservices architecture** decomposes an application into a suite of **small, autonomous services**, each built around a single business capability (e.g., Orders, Payments, Inventory), running in its own process, owning its own data store, and communicating over lightweight protocols (HTTP/REST Web APIs, gRPC, or asynchronous messaging such as RabbitMQ/Azure Service Bus). This contrasts with a **monolith**, where all modules are compiled and deployed as one unit. In the .NET ecosystem, microservices are typically built as ASP.NET Core Web API/gRPC services, containerised with Docker, and orchestrated with Kubernetes or Azure Container Apps (frameworks such as Dapr and .NET Aspire assist).

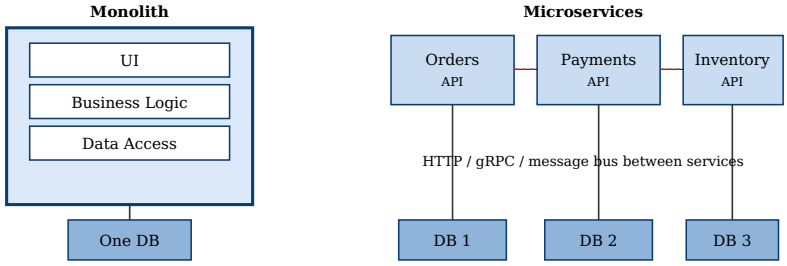


Fig 17.1: Monolith vs microservices — independent services with their own databases

Benefits

1. **Independent deployment & agility:** Each service is built, tested, and released separately — small, frequent, low-risk deployments (CI/CD) instead of redeploying the whole system for one change.
2. **Selective scalability:** Only hot services (e.g., Orders during a sale) are scaled out horizontally, saving infrastructure cost compared with cloning an entire monolith.
3. **Fault isolation / resilience:** A failure in one service (with proper patterns — circuit breakers via Polly, retries, timeouts) does not bring down the entire application.
4. **Technology & data-store freedom:** Each team can pick the best tool per service — different .NET versions, or even other languages, and polyglot persistence (SQL Server, PostgreSQL, Redis, Cosmos DB).
5. **Team autonomy & ownership:** Small teams own services end-to-end, aligned with business domains (Domain-Driven Design bounded contexts), reducing coordination overhead in large enterprises.
6. **Easier comprehension & replacement:** Small codebases are easier to understand, rewrite, or retire than a million-line monolith.

Challenges

1. **Distributed-system complexity:** Network calls replace in-process calls — latency, partial failures, retries, and timeouts must be handled everywhere; debugging spans many services.
2. **Data consistency:** Each service owns its data, so ACID transactions across services are impossible; teams must adopt **eventual consistency**, sagas, and event-driven patterns — significantly harder than a single database transaction.
3. **Operational overhead (DevOps maturity required):** Dozens of services need containerisation, orchestration (Kubernetes), service discovery, API gateways (Ocelot/YARP), configuration, and secrets management.
4. **Observability:** Centralised logging, distributed tracing (OpenTelemetry/Application Insights), health checks, and monitoring become mandatory to understand system behaviour.
5. **Testing difficulty:** Integration and end-to-end testing across independently deployed services (with versioned contracts) is more complex than testing one process.
6. **Network/security surface:** Inter-service communication must be secured (mTLS, tokens) and adds serialisation and bandwidth costs.
7. **Organisational cost:** Requires skilled teams and cultural change; for small applications the overhead outweighs benefits — "start with a monolith" is often wiser (a well-structured modular monolith can evolve into microservices later).

Conclusion: Microservices give enterprises agility, scalability, and resilience, but at the price of distributed complexity, consistency challenges, and heavy DevOps investment — adopt them when domain size, team structure, and scaling needs justify the cost.

Available on notes.arpanadhikari7.com.np

Q18. Discuss the Repository design pattern. Explain how it abstracts data access logic and promotes separation of concerns in ASP.NET applications. [8 Marks]

Answer:

Concept

The **Repository pattern** is a data-access design pattern that mediates between the business/domain layer and the data-mapping layer. A repository behaves like an **in-memory collection of domain objects**: business code asks the repository for entities (`GetAll`, `GetById`, `Add`, `Update`, `Delete`) and never deals with SQL, `DbContext`, or connection details. The repository is defined by an **interface**; concrete implementations (EF Core, Dapper, in-memory) can be swapped freely.

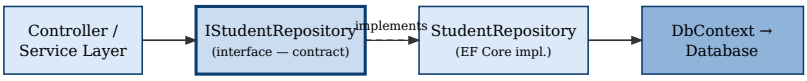


Fig 18.1: Business layer depends only on the repository interface, never on EF/SQL

Implementation in ASP.NET Core

```
// 1. Contract — what the business layer sees
public interface IStudentRepository
{
    Task<IEnumerable<Student>> GetAllAsync();
    Task<Student?> GetByIdAsync(int id);
    Task AddAsync(Student s);
    void Update(Student s);
    void Delete(Student s);
    Task SaveAsync();                       // unit-of-work commit
}

// 2. EF Core implementation — the ONLY place that knows about DbContext
public class StudentRepository : IStudentRepository
{
    private readonly SchoolContext _db;
    public StudentRepository(SchoolContext db) => _db = db;

    public async Task<IEnumerable<Student>> GetAllAsync() => await _db.Students.ToListAsync();
    public async Task<Student?> GetByIdAsync(int id)     => await _db.Students.FindAsync(id);
    public async Task AddAsync(Student s)                => await _db.Students.AddAsync(s);
    public void Update(Student s)                        => _db.Students.Update(s);
    public void Delete(Student s)                        => _db.Students.Remove(s);
    public Task SaveAsync()                              => _db.SaveChangesAsync();
}

// 3. Registration
builder.Services.AddScoped<IStudentRepository, StudentRepository>();

// 4. Controller depends only on the abstraction
public class StudentsController : Controller
{
    private readonly IStudentRepository _repo;
    public StudentsController(IStudentRepository repo) => _repo = repo;

    public async Task<IActionResult> Index() => View(await _repo.GetAllAsync());
}
```

notes.arpanadhikari7.com.np

How It Abstracts Data Access Logic

- All persistence details — LINQ-to-Entities queries, change tracking, SQL, connection handling — are **encapsulated inside the repository class**; callers use simple collection-like methods.
- The business layer is written against the **interface**, so it has no compile-time dependency on EF Core; the ORM or database can be replaced (EF → Dapper, SQL Server → PostgreSQL) by writing a new implementation only.
- Query logic is **centralised and reused** — a rule like "active students only" is defined once in the repository rather than duplicated in every controller.
- Often combined with the **Unit of Work** pattern to coordinate several repositories in a single transaction, and with a **generic repository** (`IRepository<T>`) to avoid repetition.

How It Promotes Separation of Concerns

1. **Layered responsibility:** Controllers handle HTTP, services handle business rules, repositories handle persistence — each layer has one reason to change (Single Responsibility Principle).
2. **Testability:** Controllers/services are unit-tested with a **mock repository** (e.g., Moq) — no database needed, tests run fast and deterministically.

Available on notes.arpanadhikari7.com.np

3. **Maintainability:** Schema or ORM changes are confined to the repository layer; UI and business code remain untouched.
4. **Consistency & security:** A single choke point for data access enforces validation, caching, and logging policies uniformly.

Note: Since EF Core's `DbSet` already resembles a repository, some teams use DbContext directly for simple apps; the pattern pays off most in large, test-driven, or multi-datasource ASP.NET applications.

notes.arpanadhikari7.com.np

Q19. Discuss the benefits of applying SOLID principles in software development. [7 Marks]

Answer:

SOLID is an acronym for five object-oriented design principles (popularised by Robert C. Martin) that guide developers toward software that is easy to understand, extend, test, and maintain. They are foundational to good .NET architecture (ASP.NET Core itself — DI, middleware, interfaces — is built around them).

The Five Principles

Principle	Statement	Example in .NET
S — Single Responsibility (SRP)	A class should have only one reason to change — one job.	Split `ReportService` from `ReportPrinter` and `ReportRepository` instead of one class doing calculation, printing, and saving.
O — Open/Closed (OCP)	Software entities should be open for extension but closed for modification.	New payment types added by new `IPayment` implementations — no editing of existing, tested code; middleware pipeline extends without changing the framework.
L — Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without breaking behaviour.	Any `Stream` subclass (FileStream, MemoryStream) works wherever `Stream` is expected; avoid a `Square : Rectangle` that violates setter expectations.
I — Interface Segregation (ISP)	Clients should not be forced to depend on members they do not use — prefer many small interfaces.	`IReadableRepo` / `IWritableRepo` instead of one fat `IRepository` with 20 methods; .NET's `IEnumerable`, `IComparable` are small and focused.
D — Dependency Inversion (DIP)	High-level modules should depend on abstractions, not concrete low-level classes.	Controller depends on `IMessageService`, injected by the ASP.NET Core DI container — not on `new EmailService()`.

Benefits of Applying SOLID

1. **Maintainability:** Small, single-purpose classes (SRP) localise changes — fixing or altering one behaviour cannot silently break unrelated features, lowering long-term maintenance cost.
2. **Extensibility with lower risk:** OCP lets new features be added as *new* classes/plug-ins rather than edits to stable code, so existing tested functionality remains untouched (fewer regressions).
3. **Reliability and correct polymorphism:** LSP guarantees that inheritance hierarchies behave predictably, preventing subtle bugs when derived types replace base types.
4. **Loose coupling & flexibility:** DIP + ISP decouple modules through slim abstractions, so implementations (database, logger, payment gateway) can be swapped without rippling changes — essential for evolving enterprise systems.
5. **Testability:** Dependencies expressed as interfaces are easily mocked, enabling fast, isolated unit tests — the backbone of TDD and reliable CI/CD pipelines.
6. **Readability & team productivity:** Focused classes and interfaces are self-documenting; new team members understand and modify the codebase faster, and parallel development is easier because modules interact only through contracts.
7. **Reusability:** Cohesive, decoupled components can be reused across projects and layers.
8. **Architectural foundation:** SOLID underpins patterns and architectures used in .NET — Repository, DI/IoC, middleware, Clean/Onion architecture, and microservices — making systems scalable in both code size and team size.

Conclusion: SOLID reduces rigidity, fragility, and immobility in code. The result is software that is cheaper to change, safer to extend, easier to test, and more resilient to evolving requirements — the central goals of professional software engineering.

notes.arpanadhikari7.com.np

Q20. Describe the Garbage Collection mechanism in .NET. [8 Marks]

Answer:

Concept

Garbage Collection (GC) is the CLR's **automatic memory management** service. Reference-type objects are allocated on the **managed heap**; when they are no longer reachable from any live reference, the GC reclaims their memory automatically. Developers therefore do not call `free()` / `delete`, eliminating memory leaks, double-frees, and dangling pointers common in unmanaged languages.

Working — Phases of a Collection

1. **Marking:** Starting from the application **roots** (static fields, local variables/registers of active threads, GC handles), the GC builds a graph of all *reachable* objects and marks them live.
2. **Relocating/Planning:** References to surviving objects are updated to their new addresses.
3. **Sweeping & Compacting:** Memory of unmarked (dead) objects is reclaimed, and surviving objects are **compacted** — moved together so free memory forms one contiguous block, making future allocations extremely fast (just advancing a pointer) and eliminating fragmentation.

Generational Model

The GC is **generational**, based on the observation that most objects die young:

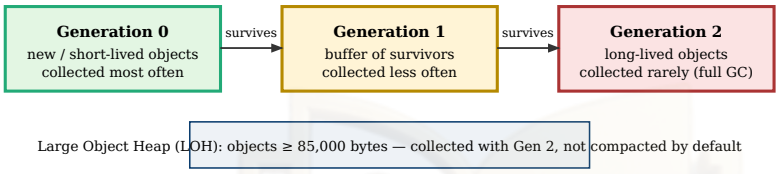


Fig 20.1: Generational managed heap in .NET

- **Gen 0:** Newly allocated small objects. Collections are frequent and very fast; most objects die here.
- **Gen 1:** Objects surviving a Gen 0 collection — a buffer between short- and long-lived objects.
- **Gen 2:** Long-lived objects (caches, static data). A Gen 2 ("full") collection also scans Gen 0/1 and the **Large Object Heap** and is the most expensive.
- Promotion: survivors of a collection move to the next generation. Collecting only young generations keeps pauses short.

When Does GC Run?

- Generation 0 fills its allocation budget (most common trigger);
- The system reports low physical memory;
- `GC.Collect()` is called explicitly (discouraged in normal code).

notes.arpanadhikari7.com.np

GC Flavours and Related Features

- **Workstation vs Server GC:** Workstation optimises responsiveness for client apps; Server GC uses one heap+thread per core for high-throughput servers (default for ASP.NET Core).
- **Concurrent/Background GC:** Gen 2 collections run largely in the background while application threads continue, minimising pauses.
- **Finalization & IDisposable:** GC frees only *managed* memory. Unmanaged resources (file handles, DB connections) are released deterministically via `IDisposable.Dispose()` / `using`; finalizers (`~Class`) are a safety net but delay reclamation, so `Dispose` + `GC.SuppressFinalize(this)` is preferred.
- **Useful API:** `GC.Collect()`, `GC.GetGeneration(obj)`, `GC.GetTotalMemory()`, `GC.WaitForPendingFinalizers()`.

Advantages

1. Eliminates manual memory bugs (leaks, dangling pointers, double frees), improving reliability and security.
2. Fast pointer-bump allocation and automatic defragmentation via compaction.
3. Generational + background design keeps pause times low while maintaining throughput.
4. Lets developers focus on business logic rather than memory bookkeeping.

Available on notes.arpanadhikari7.com.np

Q21. Explain state management techniques in ASP.NET applications. [8 Marks]

Answer:

Why State Management?

HTTP is a stateless protocol — every request is independent, and the web server forgets a page's controls, variables, and user data as soon as the response is sent. **State management** is the set of techniques ASP.NET provides to **preserve data** (user identity, shopping cart, form values) across postbacks, requests, and sessions. Techniques are divided into **client-side** (data stored on/travelling with the browser) and **server-side** (data stored in server memory or a store).

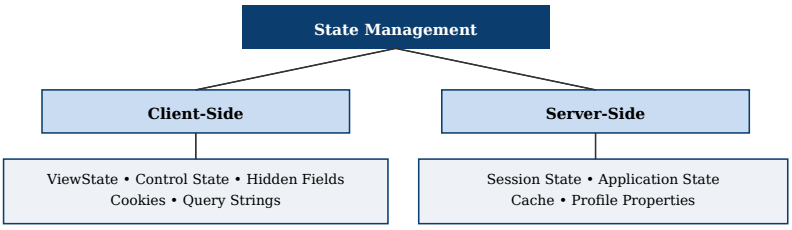


Fig 21.1: Classification of ASP.NET state management techniques

Client-Side Techniques

1. **ViewState:** Automatically serialises page/control values into the hidden, Base64-encoded `__VIEWSTATE` field, restoring them on postback of the *same page*. Easy and needs no server memory, but enlarges page size and travels on every postback. `ViewState["Count"] = 5;`
2. **Control State:** A non-disable-able portion of ViewState for data a control *must* have to function (used by control developers).
3. **Hidden Fields:** `<asp:HiddenField>` stores small values in the form; visible in page source, so never for sensitive data.
4. **Cookies:** Small text (≈4 KB) stored on the client and sent with every request to the site; persistent (with expiry) or session cookies. Good for preferences and "remember me". `Response.Cookies["theme"].Value = "dark";` Users can disable them; do not store sensitive data unencrypted.
5. **Query Strings:** Data appended to the URL — `Details.aspx?id=5&page=2`, read via `Request.QueryString["id"]`. Simple cross-page transfer, bookmarkable, but visible, limited in length, and easily tampered with.

Server-Side Techniques

1. **Session State:** Per-user data stored on the server, identified by a session ID cookie; available across *all pages* until timeout (default 20 min). `Session["Cart"] = cart;` Storage modes: **InProc** (fast, in web-server memory), **StateServer** (separate service), **SQLServer** (durable, web-farm safe), and in ASP.NET Core, **distributed caches (Redis)**. Ideal for user-specific data (login info, cart), at the cost of server memory.
2. **Application State:** Global data shared by **all users** of the application (`Application["Visitors"]`), initialised in `Global.asax`; must be locked (`Application.Lock()/UnLock()`) for safe concurrent updates. Suited to app-wide counters/config; lost on restart, not web-farm aware.
3. **Cache:** High-performance, application-wide store with expiration policies (absolute/sliding), priorities, and dependencies (e.g., invalidate when a file/DB changes) — used to avoid repeated expensive queries. ASP.NET Core offers `IMemoryCache` and `IDistributedCache`.
4. **Profile Properties:** Strongly typed, *persistent* per-user data automatically stored in a database via a provider — survives sessions and restarts.

Comparison

Technique	Scope	Stored	Lifetime	Best for
ViewState	Single page	Client (hidden field)	Across postbacks	Control values on one page
Cookies	Whole site	Client disk/memory	Set expiry	Preferences, identifiers
Query string	Between pages	URL	One request	Small non-secret parameters
Session	Per user, all pages	Server	Until timeout	Cart, logged-in user data
Application	All users	Server	App lifetime	Global counters/settings
Cache	All users	Server	Policy-based	Expensive query results

Choosing: Use client-side state for small, non-sensitive, page-level data (reduces server load); use server-side

Available on notes.arpanadhikari7.com.np

state for sensitive or larger per-user/application data (more secure, costs server resources). In ASP.NET Core, ViewState/Application state are replaced by TempData, Session, caching services, and explicit model binding.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np